Database: NoSQL (MongoDB)

Section C — Mini Capstone Project

Mini Project: QuickEats Food Delivery Backend Database

Objective: Build a complete MongoDB database for a food delivery platform called QuickEats, combining CRUD operations, query operators, update operators, aggregation pipelines, and indexing into a single cohesive system that simulates a real delivery backend.

1. Create database and collections :

```
use quickeats_db

db.restaurants.drop()
db.orders.drop()
db.customers.drop()

db.createCollection("restaurants")
db.createCollection("orders")
db.createCollection("customers")
```

2. Insert restaurants :

```
db.restaurants.insertMany([
  {
    restaurantId: "R001",
    name: "Spice Garden",
    cuisine: ["Indian", "North Indian"],
    rating: 4.7,
    deliveryTime: 30,
    location: {
      city: "Ahmedabad",
      area: "Navrangpura",
      coordinates: { lat: 23.0365, lng: 72.5660 }
    },
    menu: [
      { itemId: "M101", name: "Paneer Tikka", price: 220, category:
"Starters", available: true },
      { itemId: "M102", name: "Butter Naan", price: 60, category: "Breads",
available: true },
      { itemId: "M103", name: "Dal Makhani", price: 180, category: "Main
Course", available: true }
    ]
  },
```

```
  {
    restaurantId: "R002",
    name: "Pizza Palace",
    cuisine: ["Italian", "Fast Food"],
    rating: 4.3,
    deliveryTime: 25,
    location: {
      city: "Ahmedabad",
      area: "Satellite",
      coordinates: { lat: 23.0300, lng: 72.5100 }
    },
    menu: [
      { itemId: "M201", name: "Margherita Pizza", price: 299, category:
"Pizza", available: true },
      { itemId: "M202", name: "Farmhouse Pizza", price: 399, category:
"Pizza", available: true },
      { itemId: "M203", name: "Garlic Bread", price: 149, category: "Sides",
available: true }
    ]
  },
  {
    restaurantId: "R003",
    name: "Wok Express",
    cuisine: ["Chinese", "Asian"],
    rating: 4.6,
    deliveryTime: 35,
    location: {
      city: "Surat",
      area: "Adajan",
      coordinates: { lat: 21.1950, lng: 72.7930 }
    },
    menu: [
      { itemId: "M301", name: "Veg Hakka Noodles", price: 210, category:
"Noodles", available: true },
      { itemId: "M302", name: "Manchurian", price: 190, category: "Starters",
available: true },
      { itemId: "M303", name: "Fried Rice", price: 200, category: "Rice",
available: true }
    ]
  },
  {
    restaurantId: "R004",
    name: "Burger Hub",
    cuisine: ["American", "Fast Food"],
    rating: 4.1,
    deliveryTime: 20,
    location: {
      city: "Vadodara",
```

```
      area: "Alkapuri",
      coordinates: { lat: 22.3100, lng: 73.1750 }
    },
    menu: [
      { itemId: "M401", name: "Classic Burger", price: 199, category:
"Burgers", available: true },
      { itemId: "M402", name: "Cheese Burger", price: 249, category:
"Burgers", available: true },
      { itemId: "M403", name: "French Fries", price: 129, category: "Sides",
available: true }
    ]
  },
  {
    restaurantId: "R005",
    name: "Green Leaf Cafe",
    cuisine: ["Healthy", "Vegetarian"],
    rating: 4.8,
    deliveryTime: 28,
    location: {
      city: "Ahmedabad",
      area: "Bodakdev",
      coordinates: { lat: 23.0400, lng: 72.5100 }
    },
    menu: [
      { itemId: "M501", name: "Veg Salad Bowl", price: 250, category:
"Healthy", available: true },
      { itemId: "M502", name: "Paneer Wrap", price: 220, category: "Wraps",
available: true },
      { itemId: "M503", name: "Fresh Juice", price: 120, category:
"Beverages", available: true }
    ]
  }
])
```

3. Insert customers :

```
db.customers.insertMany([
  {
    customerId: "C001",
    name: "Aarav Shah",
    phone: "9876500001",
    email: "aarav@example.com",
    loyaltyPoints: 450,
    addresses: [
      {
        type: "Home",
        street: "12 Navrangpura Road",
        city: "Ahmedabad",
```

```
      pincode: "380009"
    },
    {
      type: "Office",
      street: "45 SG Highway",
      city: "Ahmedabad",
      pincode: "380054"
    }
  ]
},
{
  customerId: "C002",
  name: "Diya Patel",
  phone: "9876500002",
  email: "diya@example.com",
  loyaltyPoints: 320,
  addresses: [
    {
      type: "Home",
      street: "22 Adajan Road",
      city: "Surat",
      pincode: "395009"
    }
  ]
},
{
  customerId: "C003",
  name: "Rohan Mehta",
  phone: "9876500003",
  email: "rohan@example.com",
  loyaltyPoints: 180,
  addresses: [
    {
      type: "Home",
      street: "18 Alkapuri",
      city: "Vadodara",
      pincode: "390007"
    }
  ]
},
{
  customerId: "C004",
  name: "Isha Desai",
  phone: "9876500004",
  email: "isha@example.com",
  loyaltyPoints: 600,
  addresses: [
    {
```

```
        type: "Home",
        street: "8 Bodakdev",
        city: "Ahmedabad",
        pincode: "380054"
      }
    ]
  },
  {
    customerId: "C005",
    name: "Kabir Joshi",
    phone: "9876500005",
    email: "kabir@example.com",
    loyaltyPoints: 275,
    addresses: [
      {
        type: "Home",
        street: "31 Vesu",
        city: "Surat",
        pincode: "395007"
      }
    ]
  }
])
```

4. Insert orders :

```
db.orders.insertMany([
  {
    orderId: "O1001",
    customerId: "C001",
    restaurantId: "R001",
    restaurantName: "Spice Garden",
    status: "Delivered",
    orderDate: ISODate("2026-08-10T12:30:00Z"),
    items: [
      { name: "Paneer Tikka", quantity: 2, price: 220 },
      { name: "Butter Naan", quantity: 3, price: 60 }
    ],
    totalAmount: 620,
    payment: { method: "UPI", status: "Paid" },
    deliveryLog: [
      { status: "Placed", time: ISODate("2026-08-10T12:30:00Z") },
      { status: "Delivered", time: ISODate("2026-08-10T13:05:00Z") }
    ]
  },
  {
    orderId: "O1002",
    customerId: "C002",
```

```
    restaurantId: "R003",
    restaurantName: "Wok Express",
    status: "Delivered",
    orderDate: ISODate("2026-08-11T19:00:00Z"),
    items: [
      { name: "Veg Hakka Noodles", quantity: 2, price: 210 },
      { name: "Manchurian", quantity: 1, price: 190 }
    ],
    totalAmount: 610,
    payment: { method: "Card", status: "Paid" },
    deliveryLog: [
      { status: "Placed", time: ISODate("2026-08-11T19:00:00Z") },
      { status: "Delivered", time: ISODate("2026-08-11T19:40:00Z") }
    ]
  },
  {
    orderId: "O1003",
    customerId: "C003",
    restaurantId: "R004",
    restaurantName: "Burger Hub",
    status: "Preparing",
    orderDate: ISODate("2026-08-12T13:15:00Z"),
    items: [
      { name: "Cheese Burger", quantity: 2, price: 249 },
      { name: "French Fries", quantity: 2, price: 129 }
    ],
    totalAmount: 756,
    payment: { method: "Cash", status: "Pending" },
    deliveryLog: [
      { status: "Placed", time: ISODate("2026-08-12T13:15:00Z") },
      { status: "Preparing", time: ISODate("2026-08-12T13:25:00Z") }
    ]
  },
  {
    orderId: "O1004",
    customerId: "C004",
    restaurantId: "R005",
    restaurantName: "Green Leaf Cafe",
    status: "Delivered",
    orderDate: ISODate("2026-08-13T18:30:00Z"),
    items: [
      { name: "Veg Salad Bowl", quantity: 1, price: 250 },
      { name: "Paneer Wrap", quantity: 2, price: 220 },
      { name: "Fresh Juice", quantity: 2, price: 120 }
    ],
    totalAmount: 930,
    payment: { method: "UPI", status: "Paid" },
    deliveryLog: [
```

```
      { status: "Placed", time: ISODate("2026-08-13T18:30:00Z") },
      { status: "Delivered", time: ISODate("2026-08-13T19:10:00Z") }
    ]
  },
  {
    orderId: "O1005",
    customerId: "C005",
    restaurantId: "R002",
    restaurantName: "Pizza Palace",
    status: "Out for Delivery",
    orderDate: ISODate("2026-08-14T20:00:00Z"),
    items: [
      { name: "Farmhouse Pizza", quantity: 1, price: 399 },
      { name: "Garlic Bread", quantity: 2, price: 149 }
    ],
    totalAmount: 697,
    payment: { method: "UPI", status: "Paid" },
    deliveryLog: [
      { status: "Placed", time: ISODate("2026-08-14T20:00:00Z") },
      { status: "Out for Delivery", time: ISODate("2026-08-14T20:35:00Z") }
    ]
  }
])
```

5. Read queries :

Query 1 — Restaurants with rating above 4

Uses $gt and projection:

```
db.restaurants.find(
  { rating: { $gt: 4 } },
  { _id: 0, restaurantId: 1, name: 1, rating: 1, "location.city": 1 }
)
```

Query 2 — Orders with selected statuses

Uses $in and projection:

```
db.orders.find(
  { status: { $in: ["Delivered", "Preparing"] } },
  { _id: 0, orderId: 1, customerId: 1, restaurantName: 1, status: 1,
totalAmount: 1 }
)
```

Query 3 — Customers from Ahmedabad

Uses $eq and projection:

```
db.customers.find(
  { "addresses.city": { $eq: "Ahmedabad" } },
  { _id: 0, customerId: 1, name: 1, email: 1, "addresses.city": 1 }
)
```

6. Update operations :

Update 1 — Change order status using $set:

```
db.orders.updateOne(
  { orderId: "O1003" },
  {
    $set: {
      status: "Out for Delivery"
    }
  }
)
```

Update 2 — Append to delivery log using $push:

```
db.orders.updateOne(
  { orderId: "O1003" },
  {
    $push: {
      deliveryLog: {
        status: "Out for Delivery",
        time: new Date()
      }
    }
  }
)
```

Update 3 — Increment customer loyalty points using $inc:

```
db.customers.updateOne(
  { customerId: "C001" },
  {
    $inc: {
      loyaltyPoints: 50
    }
  }
)
```

Verify the updates:

```
db.orders.findOne(
  { orderId: "O1003" },
```

```
  { _id: 0, orderId: 1, status: 1, deliveryLog: 1 }
)

db.customers.findOne(
  { customerId: "C001" },
  { _id: 0, customerId: 1, name: 1, loyaltyPoints: 1 }
)
```

7. Revenue aggregation report:

```
db.orders.aggregate([
  {
    $match: {
      status: "Delivered"
    }
  },
  {
    $group: {
      _id: "$restaurantName",
      totalRevenue: { $sum: "$totalAmount" },
      orderCount: { $sum: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      restaurantName: "$_id",
      totalRevenue: 1,
      orderCount: 1
    }
  },
  {
    $sort: {
      totalRevenue: -1
    }
  }
])
```

8. Compound index:

Create the required compound index:

```
db.orders.createIndex(
  { customerId: 1, status: 1 },
  { name: "customerId_status_index" }
)
```

Verify it:

```
db.orders.getIndexes()
```

9. explain("executionStats") before the index:

```
db.orders.dropIndex("customerId_status_index")
```

Run the filtered query:

```
db.orders.find({
  customerId: "C001",
  status: "Delivered"
}).explain("executionStats")
```

10. Create the index and run explain again:

```
db.orders.createIndex(
  { customerId: 1, status: 1 },
  { name: "customerId_status_index" }
)
```

Now run exactly the same query:

```
db.orders.find({
  customerId: "C001",
  status: "Delivered"
}).explain("executionStats")
```